called *.note.stapsdt* and *.base.stapsdt* are created (sometimes, three sections including *.probes*, but that's only if we use semaphores). How does it happen? Well, if you look closely into the *<sys/sdt.h>*, the preprocessor converts the *STAP_PROBE()* macro variants into *asm* directives. This can be verified using the C preprocessor command *cpp*.

The *asm* directives help in creating these sections (*.note.stapsdt* and *.base.stapsdt*) and embedding the information about the markers into them.

The important section is *.note.stapsdt* which contains the information about all the SDT markers, like their name, provider, location in the object code, etc. Let us look at the first program (containing a single marker) shown earlier, as an example.

Compile that program:

```
$ gcc test.c -o test
```

Now, use the command *readelf* to see if those sections were created:

```
$ readelf -S ./test
```

Refer to Figure 4.

We can clearly see the section *.note.stapsdt* present. To view the contents of this section, one can either use *objdump -dt ./test* and then look for this section and its contents, or the much subtler way is to use *readelf* to read all the notes' information. SDT markers are actually *notes* with type as *stapsdt*.

Note that the ALLOC bits for the section *.note.stapsdt* are set off. That means this section won't be loaded into memory when the program is loaded to run. How does this help? If we have a large number of SDT markers being used, this might take up a large amount of space in the memory; hence, it's better not to load it.

```
$readelf -n ./test
```

The output in Figure 5 displays information about all the notes present in that program. The last one is an SDT note. We know that because we can see that the owner of that note is *stapsdt*. SDT notes have *stapsdt* as their owner. That defines their type.

Here, we find the type of note, provider name, marker name, the location in the program and some other information including a semaphore address. Let's ignore the semaphore address for now. The location shown here is 0x40053e. If, in the object code we find the instruction at the following address:

```
$ objdump -dt ./test | grep 40053e
```

…the instruction at this address will be *nop*, as shown in Figure 6.

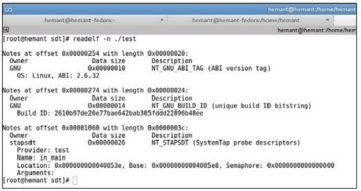

Figure 5: *stapsdt* is the SDT note

Figure 6: *nop* is stored at the location of the SDT note

So, the place at which we placed our marker has been replaced with a *nop*. This can also be seen from the assembler directives which replace the macro *STAP_PROBE()* where a *nop* instruction is placed. And since the SDT markers are replaced with *nop*'s, they are dormant (when not being probed).

Probing that location

When we probe the marker using Systemtap, the section *.note.stapsdt* is retrieved. From this section, the information about all the SDT notes/markers is also retrieved. After obtaining the information about the note, the location of the SDT marker and the executable path is sent to the kernel's *uprobes* interface. It replaces the *nop* instruction with an *int3* instruction, which is a trap (refer to Figure 7).

This interrupt is handled by a default handler in the kernel. But this is where the Systemtap again interferes. The Systemtap script we write is converted into a kernel module, which is loaded into the kernel as soon as the script is run, using *stap*.

Systemtap creates its own handler for *uprobes* (present in the kernel module) to record the necessary data like arguments, values, etc, and hence, displays it to us. So, for each hit on the event, this handler gets called and the required action is taken (the action being specified in the script we write).

I won't go into the details of Systemtap due to space limitations. One can always find documentation regarding Systemtap.

So, whenever control reaches the marker location (currently being probed) in a program, it hits *int3*. The interrupt handler for *int3* gets called and, sequentially, all the